

Document number: P1322R0  
Date: 2018-10-08  
Project: Programming Language C++  
Audience: SG1 - Concurrency and Parallelism, LEWG  
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

# Networking TS enhancement to enable custom I/O executors

## 1. Introduction

At present, the Networking TS's I/O objects -- sockets, acceptors, resolvers, and timers -- can be associated only with concrete execution contexts of type `io_context`. This paper proposes a minor change to the specification to permit the construction of these objects with arbitrary I/O executors and execution contexts. This would:

- allow implementers to support multiple "native" execution contexts, with different design and performance trade-offs; and
- simplify some use cases, by allowing users to specify their preferred executor once, at construction, rather than on each operation.

## 2. Rationale

### Construction with arbitrary execution contexts

Users and third-party libraries may implement their own execution contexts, for example to provide different scheduling guarantees, or to natively support other types of I/O objects, such as files. This proposal would allow users to create the standard I/O objects, such as sockets, to be associated with these custom execution contexts. However, although constructing the socket with a custom execution context will provide the correct, expected behaviour, there is no guarantee that it will operate as efficiently as a socket associated with a native execution context.

### Support for multiple native execution contexts

Some operating systems have multiple potential implementation strategies for the I/O objects, like sockets. For example, on Windows we can choose to have asynchronous notifications delivered via a completion port to a user-created thread, or we can have them delivered directly to the system thread pool. An implementer may want to make both of these strategies available, via the `io_context` and `system_context` execution contexts, respectively. For example:

```
io_context my_context;  
tcp::socket my_socket_1(my_context); // Implementation uses a completion port.  
  
// ...  
  
tcp::socket my_socket_2(system_executor{}); // Implementation uses the system thread pool.
```

Libraries can also provide additional native execution contexts as implementation-specific extensions.

### Convenient construction with a preferred executor

In some use cases, all asynchronous operations on a given I/O object are performed using the same executor. One common example of this is when using a strand. This proposal allows this executor to be specified once, when the I/O object is constructed. For example:

```
io_context my_context;

// ...

strand<io_context::executor_type> my_strand(my_context.get_executor());
tcp::socket my_socket(my_strand);

// ...

my_socket.async_receive(my_buffer, my_handler); // my_handler is invoked in the strand
```

### 3. Overview of changes to specification

#### Addition of Executor template parameter

A new Executor template parameter is added to the following templates:

- `basic_waitable_timer`
- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`
- `basic_socket_streambuf`
- `basic_socket_iostream`
- `ip::basic_resolver`

To enable convenient interoperability with arbitrary executors and execution contexts, the template parameter is defaulted to the executor polymorphic wrapper. For example:

```
template <class Protocol, class Executor = executor>
    class basic_socket;
```

For each of these classes, the nested type `executor_type` is altered to refer to the Executor type:

```
template <class Protocol, class Executor>
class basic_socket : public socket_base
{
public:
    using executor_type = io_context::executor_typeExecutor;
    // ...
};
```

#### Modification of I/O objects' constructors

For each of the following templates:

- `basic_waitable_timer`
- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`
- `ip::basic_resolver`

every constructor with an `io_context&` parameter is replaced with two constructors:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...
    basic_socket(io_context& ctx, const protocol_type& protocol);
    basic_socket(const executor_type& ex, const protocol_type& protocol);
    template<class ExecutionContext>
    basic_socket(ExecutionContext& ctx, const protocol_type& protocol);
    // ...
};
```

The second of these constructors shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is true, and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

## Modification of I/O objects' converting constructors

An `OtherExecutor` template parameter is added as required to the converting move constructors of the following classes:

- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`

For example:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...
    template<class OtherProtocol, class OtherExecutor>
    basic_socket(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
    // ...
};
```

This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol`, and `OtherExecutor` is implicitly convertible to `Executor`.

## Modification of I/O objects' converting assignment operators

An `OtherExecutor` template parameter is added as required to the converting move assignment operators of the following classes:

- `basic_socket`
- `basic_datagram_socket`
- `basic_stream_socket`
- `basic_socket_acceptor`

For example:

```
template<class Protocol, class Executor>
class basic_socket : public socket_base
{
    // ...
    template<class OtherProtocol, class OtherExecutor>
```

```

        basic_socket& operator=(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
    // ...
};

```

This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol`, and `OtherExecutor` is implicitly convertible to `Executor`.

## Modification of `basic_socket_acceptor` member functions

Every overload of member functions `accept` and `async_accept` with an `io_context&` parameter is replaced with two overloads:

```

template<class AcceptableProtocol, class Executor>
class basic_socket_acceptor : public socket_base
{
    // ...
    socket_type accept(io_context& ctx);
    socket_type accept(const executor_type& ex);
    template<class ExecutionContext>
    socket_type accept(ExecutionContext& ctx);
    // ...
};

```

The second of these overloads shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is true, and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

## Modification of `connect` and `async_connect` functions

An `Executor` template parameter is added as required to the `connect` and `async_connect` functions. For example:

```

template<class Protocol, class Executor, class EndpointSequence>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
                                       const EndpointSequence& endpoints);

```

## Modification of `basic_socket_streambuf`

The `basic_socket_streambuf` default constructor is modified so that it shall not participate in overload resolution unless `is_constructible<executor_type, io_context::executor_type>::value` is true.

## Modification of `basic_socket_iostream`

The `basic_socket_iostream` specification is modified so that the `executor_type` type is passed to all uses of `basic_socket_streambuf`, as required. The `basic_socket_iostream` default constructor is modified so that it shall not participate in overload resolution unless `is_default_constructible<basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>>::value` is true.

# 4. Possible implementation approach

This section is intended to provide implementation suggestions only, and is not intended to be prescriptive or exhaustive.

## Provision of default implementation

Implementations may use an execution context service as a container for the backend implementation.

```
class __socket_backend : public execution_context::service
{
    // ...
};
```

This service would be "used" by an I/O object implementation, and is automatically created on first use:

```
template <class Protocol, class Executor>
class basic_socket : public socket_base
{
public:
    using executor_type = Executor;

    // ...

    explicit basic_socket(const executor_type& ex)
    {
        auto& backend = use_service<__socket_backend>(ex.context());
        // ...
    }

    // ...
};
```

This allows the I/O object to be used with arbitrary execution contexts.

## Native implementations

A native execution context preemptively performs service creation by calling `make_service`. It can use this opportunity to pass additional constructor arguments that initialise the backend in the native mode, rather than the default:

```
class io_context : public execution_context
{
public:
    // ...

    io_context()
    {
        make_service<__socket_backend>(*this, __io_context_backend_tag{});
    }

    // ...
};
```

Alternatively, implementations may use a class hierarchy of services, and virtual functions, to select the desired behaviour:

```
class __socket_backend : public execution_context::service
{
public:
    using key_type = __socket_backend;

    // ...

    virtual void backend_function(/* ... */);

    // ...
};
```

```

class __io_socket_backend : public __socket_backend
{
public:
    // ...

    void backend_function(/* ... */) override;

    // ...
};

class io_context : public execution_context
{
public:
    // ...

    io_context()
    {
        make_service<__io_socket_backend>(*this);
    }

    // ...
};

```

In both approaches, the existing service, with its native backend, is obtained by the I/O object constructor.

## Performance impact

Specification of the polymorphic executor as the default I/O executor, while improving usability, has a non-zero impact on performance. This impact can be mitigated by having an I/O object's constructor detect its own well-known native executor types (e.g. by using `executor::target_type`). With this information, the overhead can then be limited to simple branching, rather than the memory allocations and virtual functions that would likely be required for the type-erased function objects.

Users can avoid this overhead completely by explicitly specifying the I/O executor type:

```

io_context my_context;
basic_stream_socket<tcp, io_context::executor_type> my_socket(my_context);

```

## 5. Implementation experience

This design change has been implemented in an experimental branch of Asio, for a subset of its available I/O objects.